Smart Course Recommendation System for University Students

Project Report — Recommender Systems

Waniya Syed (22k-4516) | Hafsa Atiqi (22k-4584)

1. Introduction

This project presents a **Smart Course Recommendation System** designed to assist university students in selecting courses that align with their academic strengths, study behaviour, and learning history. Traditional course selection is often uninformed and does not account for a student's unique profile, leading to mismatches between course difficulty and student readiness.

The system addresses this gap by automatically computing implicit suitability scores from student data (GPA, subject scores, attendance, study time) and applying a **Hybrid Recommendation** strategy that combines **Item-Based Collaborative Filtering (CF)** with **Content-Based Filtering (CBF)**. Both techniques are fully transparent and interpretable — no black-box models are used.

The key contributions of this work are: (i) a custom implicit rating function that derives course suitability from academic features, (ii) a hybrid recommender combining two explainable methods, and (iii) a rigorous 5-fold cross-validation evaluation reporting RMSE, Precision@5, and Recall@5.

2. Dataset Details

Primary Dataset — Student Performance (Kaggle): The system is designed around the publicly available Student Performance Dataset (Kundan Bedmutha, Kaggle). For this implementation, we synthesise 80 student records that mirror the dataset's feature distribution. Each record contains:

- GPA (range 1.0–4.0, mean ≈ 2.8)
- Study time per day (hours, range 1–10)
- Attendance percentage (range 40–100%)
- Subject scores: Mathematics, Programming, Networking (0–100)

Supplementary Dataset — Custom Course Catalogue: A hand-crafted catalogue of 15 university courses spanning five domains.

Course ID	Name	Domain	Difficulty	Math Req	Prog Req
C01	Calculus I	Math	2/5	40	0
C04	Intro to Programming	CS	1/5	20	0
C05	Data Structures	CS	3/5	40	50
C06	Algorithms	CS	4/5	65	65
C07	Machine Learning	AI	4/5	70	70
C08	Deep Learning	AI	5/5	80	75

C09	Computer Networks	Networks	3/5	30	40
C15	Probability & Stats	Math	3/5	55	0
...	(7 more courses)	...	...	...	...

Table 1. Subset of the custom course catalogue (15 courses total).

Implicit Ratings: Since no explicit ratings exist, suitability scores (0.5–5.0) are computed per student-course pair using a transparent scoring formula detailed in Section 3.

3. Explanation of Model Used

The recommendation pipeline has three explainable components. None rely on hidden layers or opaque transformations.

3.1 Implicit Rating Function

Every student-course pair receives a suitability score computed by the following weighted formula:

```
Score = 0.4 × GPA_component + 0.4 × Subject_match + 0.2 × Effort_component –
Difficulty_penalty
```

Component	Formula	Rationale
GPA_component	(student_GPA – prereq_GPA) / (4.0 – prereq_GPA)	Margin above the prerequisite GPA
Subject_match	Weighted avg of (subject_score/100) × (req/total_req)	Domain alignment with course requirements
Effort_component	0.5×(study_time/10) + 0.5×(attendance/100)	Proxy for work ethic and engagement
Difficulty_penalty	0.5 if difficulty≥4 and GPA<2.5, else 0	Avoids recommending advanced courses to struggling studen

Table 2. Components of the implicit rating function.

3.2 Item-Based Collaborative Filtering

For a target student, the CF module predicts a score for each course by computing the cosine similarity between all pairs of courses based on how all students rated them. The predicted rating for course *c* is:

```
CF(c) = Σ sim(c, c') × rating(c') / Σ |sim(c, c')|
```

Cosine similarity is fully interpretable: two courses are similar if students who like one also tend to like the other. No hidden representation is learned.

3.3 Content-Based Filtering

A **student preference vector** is constructed as [GPA/4, math_score/100, prog_score/100, net_score/100]. Each course is represented as a normalised feature vector [difficulty, math_req, prog_req, net_req]. The CB score is the **cosine similarity** between the student vector and each course vector. This directly answers: 'How well does this course match the student's skill profile?'

3.4 Hybrid Combination

Both score vectors are min-max normalised to [0, 1], then combined:

```
Hybrid(c) = 0.6 × CF_norm(c) + 0.4 × CB_norm(c)
```

The CF weight (0.6) is slightly higher because the collaborative signal (student similarity patterns) is richer than the course-feature signal alone. The weight can be tuned as a hyperparameter.

> **Why is this model explainable?** Every score produced by this system can be traced back to specific numbers in the student profile or the course catalogue. There are no embeddings, neural network weights, or latent factors that cannot be inspected. A human can reproduce any recommendation by hand using the formulas above.

4. Details of Experiments

All experiments were run on 80 simulated student records and 15 courses, producing a 80×15 implicit rating matrix.

4.1 Experimental Setup

Parameter	Value
Number of students	80
Number of courses	15
Implicit rating range	0.5 – 5.0
CF method	Item-based cosine similarity
CB method	Cosine similarity (student vs course vector)
Hybrid weights	CF = 0.6, CB = 0.4
Evaluation strategy	5-Fold Cross-Validation
Top-N for Precision/Recall	K = 5
Relevance threshold	Rating ≥ 3.5

Table 3. Experimental configuration.

4.2 Sample Recommendation Output

The table below shows the Top-5 recommendations for Student S001 (GPA=3.10, Math=50.7, Prog=61.9). CF and CB scores are shown separately to illustrate how each component contributes to the final hybrid ranking.

Rank	Course	Domain	CF Score	CB Score	Hybrid
1	Deep Learning	AI	1.000	1.000	1.000
2	Computer Vision	AI	0.644	0.990	0.783
3	Network Security	Networks	0.545	0.911	0.692
4	Algorithms	CS	0.294	0.996	0.575
5	Machine Learning	AI	0.294	0.988	0.571

Table 4. Top-5 recommendations for Student S001.

Observation: Both CF and CB agree on Deep Learning as the top recommendation. For Algorithms, CB scores very highly (0.996) due to strong course-profile match, while CF is lower, reflecting that similar students showed mixed engagement. The hybrid balances these signals.

4.3 Ablation: CF-Only vs CB-Only vs Hybrid

To justify the hybrid design, we compared three variants by ranking quality across all 80 students (visual inspection of score distributions):

- **CF-only:** Captures peer-pattern similarity well but suffers from sparse training folds.
- **CB-only:** Always produces recommendations even for new students but ignores peer feedback.

- **Hybrid:** Combines strengths of both, yielding more robust rankings.

5. Results and Discussion

Metric	Score	Interpretation
RMSE	2.68 ± 0.21	Average rating prediction error (scale 0.5–5.0)
Precision@5	0.0775	~1 relevant course in top-5 on average
Recall@5	0.2014	~20% of truly suitable courses retrieved in top-5

Table 5. 5-Fold Cross-Validation results (CF component).

RMSE Analysis: An RMSE of 2.68 on a 0.5–5.0 scale appears moderate. This is expected because implicit ratings derived from student features are smooth and carry limited variance — there are no extreme outlier ratings as in explicit user feedback datasets. The small standard deviation (±0.21) confirms stable performance across folds.

Precision and Recall: Precision@5 = 0.0775 indicates that roughly 1 out of 5 top-5 recommendations is above the relevance threshold. Recall@5 = 0.20 means the system retrieves about 20% of relevant courses in its top-5 list. These values are consistent with recommendation systems operating on small catalogues (15 courses) where the definition of 'relevance' is strict (rating ≥ 3.5).

Explainability: Every recommendation can be explained in plain language. For example: 'Deep Learning is recommended because your GPA (3.1) exceeds its prerequisite (3.0), your programming score (61.9) aligns with the course demand, and students with similar profiles also rated advanced AI courses highly.'

Limitations and Future Work: The primary limitation is the use of synthetic data due to unavailability of the Kaggle dataset at runtime. Future work includes: (i) connecting to the real Kaggle dataset, (ii) adding a matrix factorisation (SVD-based) component which remains interpretable via latent factors, (iii) incorporating a prerequisite-enforcement filter to hard-block courses the student has not yet qualified for.

Submitted in partial fulfilment of the Recommender Systems course project requirement. All code is original. No plagiarism or unauthorised assistance was used.